

MediaPipeHand — AI 비전 피드백 페루프 EMS 제어 시스템

프로젝트 진행 내용 정리 보고서

대상 경로: C:\Users\brigh\OneDrive\바탕화면\mediapipehand | 작성일: 2026-08-28 | Claude Code 소스 분석 기반 자동 정리

목차

- 프로젝트 개요
- 시스템 구성 (파일별 역할)
- 핵심 기술 — 손 관절각 계산
- 앱 모드 3가지
- 페루프 제어 로직
- 하드웨어 연동 (Arduino / Web Serial)
- 데이터 저장 (localStorage / Supabase)
- 자연어 명령 해석 (규칙 기반 + AI)
- 진행 이력 / 버전 추이
- 미완성 항목 · 주의사항
- 실행 방법

1. 프로젝트 개요

이 프로젝트는 **MediaPipe Hand Landmarker**(손 관절 인식) 기반 컴퓨터 비전과 **EMS**(전기근육자극, **Electrical Muscle Stimulation**) 하드웨어(Arduino 연동)를 결합한 **AI 비전 피드백 페루프(closed-loop) 제어 데모 시스템**입니다.

"왼손으로 원하는 동작을 만들어 캡처(스냅샷)하면 그게 목표가 되고, 오른손(EMS 연결 대상)이 카메라로 측정되며 그 목표에 가까워지도록 EMS 제어값(0~100)이 자동 조절됩니다." — [index.html:185-198](#)

- 왼손(SOURCE / TARGET)** — 목표 동작을 지정하는 손 (카메라로만 인식, EMS 패치 없음)
- 오른손(ACTUAL)** — 실제 EMS 자극이 가해지는 손 (카메라로 측정 + 전기자극 인가)

카메라가 두 손의 손가락 굽힘 정도를 실시간 측정 → 목표와 실제의 오차 계산 → 비례제어(P control)로 EMS 출력 자동 조절 → 목표 도달 시 자동 정지/유지되는 구조입니다.

UI의 사전정의 동작이 "수저 들기 보조", "이두운동(재활/보조)" 등인 점, 참가자별 개인화 반응 곡선을 저장하는 "회원번호" 시스템, "참가자 프로필(Supabase)" 용어 등으로 미루어 **손 움직임에 제약이 있는 사용자를 위한 EMS 기반 손동작 보조·재활 연구 프로토타입**으로 추정됩니다. 코드 내 "교수님이 말씀하신 '사용자별 근육 반응 학습'"(`app.js:2816`) 주석으로 볼 때 지도교수가 있는 학부/대학원 연구 과제(캡스톤 혹은 관련 프로토타입)로 보이며, 자연어/AI(Gemini) 명령 해석까지 실험적으로 결합한 것으로 보아 **"AI 비전 + 자연어 인터페이스 + 페루프 EMS 제어"** 통합 검증이 목적으로 판단됩니다.

2. 시스템 구성

파일	크기 / 날짜	역할
<code>index.html</code>	37,181 B / 08-27	웹 데모 UI 전체 (모드 선택, 카메라 캘리브레이션, 안전설정, 행동 카드, 페루프 모니터링, 그래프, CSV 로그, 개인화 캘리브레이션)
<code>app.js</code>	149,841 B / 08-27 / 3,410 줄	메인 로직 전체 — MediaPipe 연동, 각도 계산, 명령 파서, 3가지 앱 모드, 페루프 컨트롤러, 안전관리자, Web Serial 통신, Supabase, localStorage, 그래프/CSV

hand_camera.py	6,044 B	OpenCV+MediaPipe 파이썬 프로토타입 — 웹캠으로 5손가락 굽힘각을 실시간 표시 (EMS 연동 없음, app.js 각도식의 원형)
hand_landmarker.task	7.8 MB	MediaPipe 손 랜드마크 인식 모델 (Python 버전용 로컬 모델, 웹 버전은 CDN 로드)
run_server.bat	142 B	<code>python -m http.server 8000</code> 로컬 정적 서버 실행 배치파일
ems_demo_standalone.html	10.5 MB / 08-24	더 오래된 초기 버전을 하나의 파일로 번들링한 독립 실행형 데모. 인식 모델을 base64로 내부 임베드해 서버 없이 열 수 있으나, 인식 엔진 JS는 여전히 CDN에서 로드 (완전 오프라인 아님)
mediapipehand\ (내부 폴더)	08-23	더 이전 개발 단계의 구버전 스냅샷 (3번 참고)

⚠ 발견된 문제 — 기존 PDF 산출물 손상

폴더에 있던 EMS_project_report.pdf 와 EMS_페루프_제어_시스템_개발기록.pdf 두 파일은 바이트 단위로 완전히 동일하며, 실제 보고서 내용이 아니라 "파일을 찾을 수 없음 (ERR_FILE_NOT_FOUND)" 브라우저 오류 화면을 그대로 인쇄(Print to PDF)한 결과물입니다. PDF 메타데이터의 Creator가 HeadlessChrome/Edg 151 로 찍혀 있어, 이전에 헤드리스 브라우저로 요약 페이지를 PDF화하려다 파일 경로 인코딩 문제로 실패한 결과로 추정됩니다. 즉 두 파일은 사실상 빈 문서이며 이번에 이 보고서로 대체됩니다.

3. 핵심 기술 — 손 관절각 계산

Python(hand_camera.py)과 JS(app.js) 양쪽에 **동일한 공식**이 구현되어 있습니다.

3D 관절각 계산

- MediaPipe의 **world landmarks**(손목 기준 실제 3D 좌표, 미터 단위)를 사용
- 세 점 A(기준)–B(중심 관절)–C(다음 관절)에 대해 벡터 $v1=A-B$, $v2=C-B$ 의 내적/코사인으로 사이각 계산
- `angle = acos(clip(v1·v2/(|v1||v2|), -1, 1))` (도 단위 변환)
- 굽힘량 = $180 - \text{angle}$ (편 상태 $\approx 180^\circ \rightarrow$ 굽힘 0°), 음수는 0으로 클램프
- 손가락 1개당 관절 3개의 굽힘량 합산, 0~270도로 클램프

웹 버전(app.js)은 **엄지를 제어에서 제외**합니다 — CMC 관절 구조가 달라 동일 공식이 정확하지 않기 때문 (app.js:38-51). **제어(Index/Middle/Ring/Little 4손가락)**와 **표시 전용(엄지 포함 5손가락)**을 명확히 분리했습니다.

노이즈 완화

- Python 버전: `deque(maxlen=5)` 단순 이동평균 (5프레임)
- JS 버전(`MedianEmaFilter` , app.js:224-244): 최근 5프레임의 **중앙값**을 구한 뒤 지수이동평균(EMA, $\alpha=0.35$) 적용 — 순간적 튜(outlier)에 더 강건

% 환산은 캘리브레이션(피짐/구부림) 값 대비 선형 보간이며, 오른손 기준 캘리브레이션을 왼손 값 환산에도 재사용합니다(두 손 유연성이 비슷하다는 가정).

4. 앱 모드 3가지

원래 "거울 모드"가 왼손 캡처/명령/프리셋 카드였으나, **사용자 요청으로 해당 UI를 제거**하고 "행동 보조 모드"가 그 역할 일부를 흡수하는 형태로 재구성되었습니다. 3개 모드는 상호 배타적이며, 전환 시 이전 모드의 모든 제어를 강제로 0 정지시켜 채널 충돌을 방지합니다.

① 거울 모드 (mirror) — 페루프

- **행동 버튼**(컵 잡기 / 따르기 / 흔들기 / 팔 굽히기 / 물건 놓기)으로 사전정의 목표 % 로드
- **실시간 왼손 연동**: 매 프레임 왼손 현재 굽힘값을 그대로 오른손 목표로 계속 갱신 전송
- 카메라 오차 보정이 있는 **페루프** 방식

② 행동 보조 모드 (action) — 오픈루프

"채널1/채널2에 직접 입력한 고정 전류를 오픈루프로 출력 (카메라 오차 보정 없음)" — index.html:230-231

- **수저 들기 보조**: 채널1 2초 램프업 → 채널2 3초 램프업, 순차 유지
- **이두운동**: 채널1은 2초 램프 후 유지, 채널2는 "3초 상승→1초 유지→3초 하강→0.5초 휴식"을 지정 횟수 반복
- **회원 개인화**: 회원번호로 채널1/채널2/반복횟수를 localStorage에 저장·불러오기
- **음성 명령**: Web Speech API로 "수저들기보조"/"이두운동" 두 단어만 인식해 바로 실행

③ 개인화 모드 (personalization) — 수동 램프업

"선택한 채널의 전류를 서서히 올리다가 키보드 'A'로 정지 (역치/반응 확인용)" — index.html:232-233

별도로 **개인화 캘리브레이션(자동 스윙)** 기능이 있어, 20/40/60/80/100 등 여러 자극값을 순서대로 자동으로 주고 실제 반응(% 굽힘)을 측정해 개인별 선형회귀 모델을 만듭니다. (SWEEP_HOLD_MS=4000ms 유지, SWEEP_REST_MS=2500ms 휴식, SWEEP_RESEND_MS=900ms 재전송 주기)

전체 상태값: STANDBY / IDLE / WAITING_FOR_HAND / INCREASING / DECREASING / HOLDING / SUCCESS / LOCKED / SAFETY_STOP / SWEEP_HOLD / SWEEP_REST

5. 페루프 제어 로직

핵심 함수: `updateController(currentPercent, now)` — app.js:2490-2574

- **오차**: $error = targetPercent - currentPercent$
- **허용범위 내**($\pm tolerancePercent$, 기본 5%): 마지막 세기를 계속 유지(연속 출력). successHoldSeconds(기본 1.5초) 이상 유지되면 **LOCKED**(도달 완료) 전환, 해제 버튼 전까지 자극 유지
- **오차 큼**: controlPeriodMs(기본 6000ms, $\div 2.5 = 2400ms$) 주기로만 비례(P) 제어 스텝 계산 — 매 프레임 전송 시 시리얼 큐 적체로 지연되는 문제가 실측되어 스로틀 처리
- **2채널 방향 라우팅**: $error > 0$ (덜 구부러짐)→채널1(굴근), $error < 0$ (더 구부러짐)→채널2(신근). 채널 전환 시 0부터 램프업(길항근 동시자극 방지)
- **게인**: $kpUp = 0.02$ (채널1, 조심스럽게 증가) / $kpDown = 0.3$ (채널2, 빠르게 회수) / $maxStepUp = 1$ / $maxStepDown = 8$
- 내부 작업값은 float로 누적 — 정수 반올림 시 미세 증가분이 사라져 목표 근처서 멈추는 버그 방지
- controllerIntensity(0~100)는 config.safety.maxIntensity로 클램프 후 하드웨어 전송; 화면엔 클램프 전/후 값을 분리 표시

안전 / 위치독

- **runtimeCheck()**: 시리얼 연결 끊김, 오른손 놓침(연속 10프레임 미검출), 연속 자극시간 초과(기본 10초), 전체 실험 시간 초과(기본 300초=5분, 카메라 켜 시점부터 카운트) 감시
- **비상정지**: ESC/SPACE 키, 화면 비상정지 버튼, **브라우저 탭 백그라운드 전환 시 자동 트리거**
- **safetyTripped**: 한 번 트립되면 "안전 정지 해제" 버튼 누르기 전까지 모든 모드가 즉시 재차단됨
- **LOCKED 상태**: 일반은 6초 간격 전송이지만 LOCKED(유지)일 땐 400ms 간격으로 촘촘히 재전송

6. 하드웨어 연동 (Arduino / Web Serial)

`class WebSerialLink` (app.js:3040-3223) — Web Serial API로 Arduino와 텍스트 프로토콜 통신.

- **Baud rate:** 19200
- **연결 절차:** requestPort() → open → 2초 대기(보드 리셋 안정화) → PING 최대 3회 재시도, PONG 대기(핸드셰이크) → 성공 시 400ms마다 백그라운드 PING(연결 유지)
- **명령 프로토콜:** `ARM,<ch>` / `SET,<ch>,<intensity>,<durationMs>` / `STOP,<ch>` / `STOP_ALL`
- **TTL 기반 재전송:** SET 명령에 duration(TTL, 기본 1500ms, 최대 5000ms)을 실어 보내 자극 끊김 방지. 지속 출력 필요 시 900ms 등 짧은 주기로 재전송
- **채널 전환 안전장치:** 채널 변경 시 새 채널을 켜기 전 이전 채널을 명시적으로 0 OFF 전송
- 브라우저 요구사항: Chrome 또는 Edge (미지원 시 연결 버튼 비활성화)

7. 데이터 저장

localStorage (브라우저/컴퓨터 로컬)

키	내용
<code>emsWebConfig</code>	프리셋/제어 게인/안전값/시리얼 설정
<code>emsWebCalibration</code>	퍼짐/구부림 캘리브레이션(손가락별 각도)
<code>emsWebAdaptiveModel</code>	로그 기반 자동 회귀 모델(slope/intercept)
<code>emsWebPersonalization</code>	개인화 스윙 결과 점들 + 회귀계수
<code>emsWebAiConfig</code>	AI(Gemini) 사용 여부/API 키/모델명
<code>emsWebMembers</code>	회원번호별 채널1/채널2/반복횟수
<code>emsWebParticipantId</code>	Supabase 참가자 ID 캐시

Supabase 클라우드 (calibration_profiles 테이블)

URL/anon key가 app.js 상단에 하드코딩되어 있습니다. 코드 주석에서 명시적으로 리스크를 밝힙니다:

"anon key는 클라이언트에 노출돼도 되는 공개 키입니다 (RLS 정책으로 접근 범위를 제한). 로그인 없이 참가자 ID 문자열 하나로만 구분하는 프로토타입 구조라, **같은 ID를 아는 사람은 그 데이터를 읽고 덮어쓸 수 있다**는 점은 감안하세요." — app.js:17-19

목적은 참가자가 다른 컴퓨터/브라우저로 옮겨도 캘리브레이션·개인화 데이터를 이어서 쓸 수 있게 하는 것입니다. **현재 상태:** 관련 함수는 코드에 남아있지만, 이를 쓰는 UI 카드(참가자 프로필)와 이벤트 바인딩은 "나중에 다시 추가할 예정"으로 **전부 주석 처리되어 비활성화된** 상태입니다.

8. 자연어 명령 해석

규칙 기반 (rule-based) — 현재 비활성 UI

`interpretCommand(text)` (app.js:259-298) — `command_interpreter.py`를 JS로 포팅, LLM 미사용.

- 정지어(정지/멈춰/스탑/stop), 종료어(종료/끝내/quit/exit) 처리
- "NN%" / "NN퍼센트" / "NN프로" 정규식 직접 지정

- 프리셋 매칭: 살짝/조금만→light, 최대한/세게→strong, 반쯤/절반→half

AI(Gemini) 경로 — 현재 비활성 UI

`interpretCommandWithAI(text)` (app.js:1492-1599) — 브라우저에서 Google Gemini API 직접 호출(백엔드 서버 없음), 기본 모델 gemini-3.7-flash.

- responseSchema로 JSON 강제 출력(action/target_percent/message)
- "탁구공을 칠 정도로" 같은 비유도 크기/모양 추론해 % 값으로 매핑하도록 프롬프트 설계
- API 키는 브라우저 localStorage에만 저장, 이 컴퓨터에서 직접 Gemini API로 전송
- 호출 실패/키 없음/파싱 실패 시 항상 규칙 기반으로 조용히 대체(fallback)

현재 상태: 이 두 경로와 관련 UI 카드(캡처/명령입력/프리셋) 전체가 index.html/app.js에서 통째로 주석 처리되어 현재 화면상 사용 불가능합니다. 함수 자체는 남아있어 주석만 해제하면 복구 가능하도록 설계되어 있습니다.

9. 진행 이력 / 버전 추이

버전	app.js	index.html	특징
내부 <code>mediapipehand\mediapipehand\</code> 이전 스냅샷	84,194 B (08-23)	21,267 B (08-23)	왼손 캡처/자연어 명령/AI(Gemini)/프리셋 카드가 활성화 된, "거울 모드"만 있던 단일 모드 버전으로 추정
바깥 최신 버전 현재	149,841 B (08-27)	37,181 B (08-27)	3가지 모드(거울/행동 보조/개인화) 체계로 재구성, 거울 모드 UI참가자 프로필 UI는 코드에 남긴 채 비활성화
<code>ems_demo_standalone.html</code>	10.5 MB (08-24, 양쪽 폴더 동일)		3모드 체계 이전, 모델 파일까지 base64로 통째로 임베드한 독립 배포용 스냅샷 — 현재 index.html/app.js와 세대가 다름

변경 이력은 코드 주석이 스스로 설명하고 있습니다:

"예전 '거울 모드'(왼손 캡처/자연어 명령/프리셋/AI 해석) -- 사용자 요청으로 제거하고 '행동 보조 모드'가 새 '거울 모드'가 됨. 통째로 주석 처리해서 보관, 나중에 필요하면 이 주석만 풀면 그대로 복구된다." — index.html:329-332

"참가자 프로필(Supabase)은 나중에 다시 추가할 예정이라 지금은 UI에서 뺐습니다 (app.js에 관련 함수는 남겨둠)." — index.html:223-224

10. 미완성 항목 · 주의사항

1. **행동 버튼 목표값이 전부 placeholder** — 컵 잡기 55%, 따르기 45%, 흔들기 60%, 팔 굽히기 70%, 물건 놓기 5% 등은 실측이 아닌 임시 수치. 손가락별 개별 목표(targetPerFinger)는 전부 null로 미구현.
2. **캘리브레이션이 컴퓨터/브라우저 간 공유 안 됨** — localStorage 기반이라 새 컴퓨터마다 펌핑/구부림 재측정 필요. Supabase로 해결 시도했으나 현재 비활성.
3. **Arduino 물리 연결 필요** — 실제 전기자극 테스트는 USB 연결 필수 (기본은 연결 없이도 시뮬레이션 동작).
4. **Supabase anon key 노출** — 클라이언트에 하드코딩, RLS만으로 접근 제한 (개발자가 스스로 캐비어트 명시).
5. **Web Serial API 브라우저 제약** — Chrome/Edge만 지원, 로컬 서버(run_server.bat) 구동 권장.
6. **연속 자극 자동정지 제한(기본 10초)** — LOCKED 유지 중에도 자주 걸리므로 필요시 사람이 직접 값 조정 필요.
7. **모드별 총 소요시간이 전체 안전 제한(5분)을 넘을 수 있음** — 경고는 하지만 강제로 막지는 않음.

8. 거울 모드 관련 UI 전부 비활성화 — 자연어 명령/AI 해석/원손 캡처/프리셋/참가자 프로필 카드가 현재 사용 불가.
9. 이두운동 하강/휴식 시간이 임의 결정 — "명시 요구사항이 없어 상승 시간과 대칭(3초)/짧은 휴식(0.5초)으로 우선 설정, 필요시 값 조정 가능"이라고 코드 주석에 명시.
10. **kpDown** 게인 재튜닝 필요 가능성 — 채널2가 실제로 펴는 근육을 담당하게 되며 설계 의도와 달라져, 실제 반응을 보고 재조정이 필요할 수 있음.
11. 기존 PDF 산출물 손상 — EMS_project_report.pdf / EMS_페루프_제어_시스템_개발기록.pdf 두 파일 모두 내용 없이 브라우저 오류 페이지만 인쇄되어 있었음 (본 보고서로 대체).

11. 실행 방법

방법 1 (추천, 설치 없이 바로 열기)

`ems_demo_standalone.html` 더블클릭 → Chrome/Edge로 열기. 인터넷 연결 필요(인식 엔진 온라인 로드), 카메라/Arduino(USB 시리얼) 권한 허용. 단, 이 파일은 08-24 시점 구버전(단일 거울 모드) 스냅샷이므로 최신 3모드 버전이 아닙니다.

방법 2 (최신 버전 — index.html/app.js — 사용 시 필요)

1. Python 설치 (python.org, "Add to PATH" 체크)
2. `run_server.bat` 더블클릭 → "로컬 서버를 http://localhost:8000 에서 실행합니다" 확인
3. 브라우저에서 `http://localhost:8000` 접속
4. 종료 시 검은 창에서 Ctrl+C

공통 주의사항

- 처음 여는 컴퓨터라면 카메라 펌/구부림 초기값(캘리브레이션)을 다시 측정해야 함
- 실제 전기자극 테스트에는 Arduino가 그 컴퓨터에 USB로 연결되어 있어야 함

Claude Code가 프로젝트 소스코드(app.js, index.html, hand_camera.py 등)를 직접 읽어 정리한 보고서입니다. · 2026-08-28